Corso di Laurea in Ingegneria Informatica - Sede di Mantova

Fondamenti di Machine Learning

Enrico Girardi
MAT. 128523

Esame del 06/09/2024

Abstract
Progetto di Machine Learning per la previsione di una malattia renale sulla base dei dati forniti dall'ospedale, basato sul dataset Chronic Kidney Disease.

DATASET

Il dataset di riferimento è preso dal sito **UC Irvine Machine Learning Repository**, e si tratta del dataset Chronic Kidney Disease reperibile qui:
https://archive.ics.uci.edu/dataset/336/chronic+kidney+disease
condiviso con licenza Creative Commons Attribution 4.0 International (CC BY 4.0).

STRUTTURA

FILE
- main: file principale da dove lanciare il progetto
- open.py: classi relative all'importazione ed alla conversione del dataset .arff in .csv
- costanti.py: definizione di valori constanti
- eda.py: funzioni relative alla Exploratory Data Analysis
- classes.py: funzioni di utilità generale
- modelli.py: funzioni relative alla generazione di modelli di valutazione
- grafici-py: funzioni relative alla generazione di grafici

FOLDER
- csv: contiene il dataset iniziale in .arff e le sue trasformazioni in .csv
- report: contiene i report generati dal progetto relative alle metriche dei modelli
- img: contiene i grafici (che vengono salvati automaticamente in questa cartella)
- test_csv: contiene files .csv generati durante il processo per una ulteriore verifica dei dati

IMPORTAZIONE DEL DATASET

Una volta scaricato il dataset originale da in **formato .arff :**

- Viene **convertito** il formato **.arff nel formato .csv** tramite apposita funzione.
- Contestualmente viene risolto un problema del dataset contenente alcuni valor**i \t** che causano difetti su alcune righe, generando una colonna in più e facendo slittare alcuni valori alla colonna successiva.

SPLIT DEL DATAFRAME

Si divide il CSV unico in in 2 csv distinti test_csv,csv e training.csv con RATIO = 25 / 100.

EXPLORATORY DATA ANALYSIS - EDA

Si lanciano le classiche funzionalità di stampa a video per l'analisi della struttura del data frame per individuarne le caratteristiche principali e capire come procedere.

1) ANALISI PERSONALIZZATA

Si creano delle funzioni per l'individuazione di *feature numeriche* e *feature categoriche* (data type: object), per la visualizzazione dei dati contenuti nelle colonne delle *feature categoriche*, per il *conteggio dei valori unici.*

2) DATA TYPE

Alcuni valori annoverati fra le categorical features sono numeri ma **non sono codificati** come tali, vengono quindi convertiti in Data Type numerico.

3) REFUSI

Dall'analisi visiva si evince la **presenza di refusi** in alcuni valori, ad esempio: **' yes', '\tno', 'ckd\t', '\tyes'** e vengono sostituiti in modo da uniformare questi valori scritti in modo errato agli altri presenti nella feature.

4) STAMPA DEI GRAFICI:

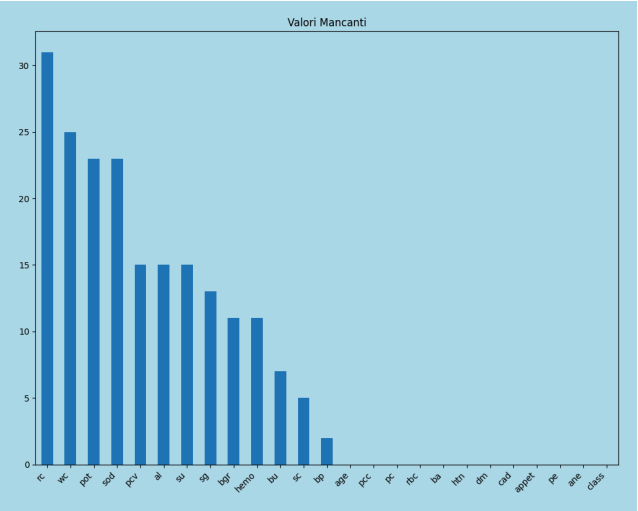
Vengono stampati:

- **un pieplot** della *target feature*: dal grafico si evince che si tratta di un dataframe **bilanciato**.
- un **countplot** delle *feature categoriche*, dal quale si evince che alcuni valori sono rappresentati dal **carattere** '**?**' ma la loro *incidenza è molto bassa*, si possono sostituire, come vedremo in seguito.
- un **distplot** delle *feature numeriche* per analizzarne le distribuzioni.
- una **heatmap** per la correlazione delle feature.

5) MODIFICA DEI VALORI CATEGORICI

Si evince anche che i valori categorici hanno al massimo 3 valori distinti, di cui uno è il **carattere '?' .** Conviene quindi trasformarli.
Il **LabelEncoder()** dà scarsi risultati (lo si evincerà in seguito in modo empirico, analizzando gli score finali), si prova quindi a **mappare manualmente** le feature categoriche, che sono solamente 11. Contestualmente quindi si forza anche la *sostituzione del carattere '?'* con un **NaN** (pd.NA) che andremo in seguito a sostituire con un'altra funzione.

6) MISSING VALUES



Dalla stampa del grafico *missing values* si evince che le colonne *'rbc', 'rc','wc','pot','sod'* **hanno molti dati mancan**ti. Dal grafico **heatmap** si evince che 'rbc', 'rc' e 'sod' hanno una **buona correlazione** con la target feature.
In fase successiva, eseguendo diversi test, si proverà che eliminando solamente 'rc' e 'sod' *abbassa molto lo score*, *eliminandole tutte e quattro si hanno prestazioni migliori*, quindi si deciderà di rimuoverle in quanto empiricamente si capirà, valutando di volta in volta gli scores per ciascun test, che troppi valori mancanti causano un modello incerto se inseriamo una sostituzione con valori random, e punteggi troppo bassi se usiamo la moda.

7) RIMOZIONE VALORI NaN
Si provano diversi metodi anche per la rimozione dei valori NaN, inizialmente si prova con una sostituzione con dei valori random presi dalla colonna (random.sample), e si proveranno tutte le combinazioni fra features numeriche, feature categoriche e sostituzione di tipo random, con la media o con la moda, ma in fase di test, basandosi sugli score ottenuti, si evincerà che si ottengono *risultati migliori* sostituendo sia le features numeriche sia feature categoriche con la *moda*, contestualmente anche alla rimozione delle colonne di cui argomentato al punto precedente.

[Nota: Questi tuning naturalmente sono stati rifatti più volte a posteriori, a modello già operativo e confrontando i risultati fra loro.]

8) RIMOZIONE DUPLICATI
Si lancia una routine per la rimozione di eventuali righe duplicate (non essendoci colonna id)

9) VERIFICA DATATYPE
Si effettua una verifica finale per verificare che ora tutte le features siano diventate numeriche.

10) FEATURE SELECTION E DEFINIZIONE DI X,y
Rimozione delle colonne 'rbc', 'rc','wc','pot','sod', 'class' da X, definizione di y con feature 'class'.

TRAIN E TEST

Una prima parte del processo viene eseguita sul **dataframe di training**, che sarà addestrato per ottenere i parametri migliori, dopodichè i risultati saranno passati alla funzione finale che li applicherà al **dataframe di test** e si otterranno i dati finali per i rispettivi modelli.

STACKING ENSEMBLE

Si sviluppano principalmente una funzione **CROSS VALIDATION K-FOLD** ed una **GRID SEARCH** che serviranno per la funzione principale dello STACKING ENSEMBLE.

SELEZIONE DEI MODELLI E DEGLI IPERPARAMETRI

Vengono selezionati i seguenti modelli: **Logistic Regression, KNeighbors, SVC, Decision Tree** e il **Multi Layer Perceptron classifier**, in quanto rappresentano categorie molto diverse di affrontare una task di classificazione, dalla classica Regressione Logistica, ai metodi non parametrici come il Nearest Neighbor e gli Alberi di Decisione, per arrivare infine alle Support Vector Machine (C-Support Vector Classification - con C che ci permette di regolarizzare il margine), e le reti neurali, ciascuno con funzionamento completamente diverso e quindi cerchiamo di affrontare il task di classificazione in diversi modi diversi fra loro per caratteristiche e funzionamento.

DOCUMENTAZIONE SUGLI IPERPARAMETRI

Per la ricerca degli iperparametri si è consultata la documentazione online:

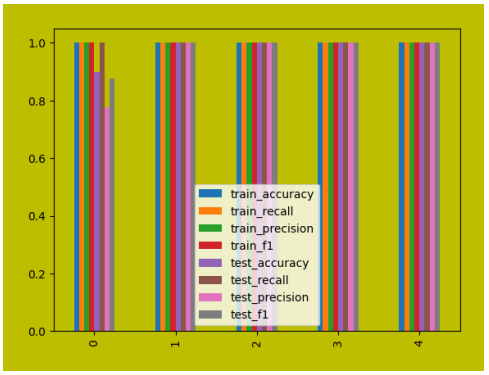
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html
- https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html
- https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
- https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier

SCHEMA DEI MODELLI CON RELATIVI IPERPARAMETRI SCELTI:

LogisticRegression	{'**penalty**': ['l1', 'l2', 'elasticnet'],'**C**': [1e-5, 5e-5, 1e-4, 5e-4, 1],'solver': ['saga'],'class_weight': ['balanced'] }
KNeighbors	{'**weights**': ['uniform', 'distance'],'**n_neighbors**': list(range(1, 12, 2)),'**algorithm**':['auto', 'ball_tree', 'brute', 'kd_tree', 'auto']}
SVC	{'**C**': [1e-2, 1, 1e1, 1e2], '**gamma**': [0.001, 0.0001],'**class_weight**': ['dict','balanced'] }
Decision Tree	{'**criterion**': ['gini', 'entropy','log_loss'],'**max_features**': ['sqrt', 'log2'],'**class_weight**': ['balanced'] }
Multi Layer Perceptron	{'**solver**': ['lbfgs', 'sgd', 'adam'], '**activation**': ['tanh', 'relu'] }

GRID SEARCH E CROSS VALIDATION

Viene effettuata una grid search fra le combinazioni dei modelli ed i loro iperparametri e viene restituita una lista con i best score degli iperparametri per ciascun modello. Viene poi eseguita una **Cross Validation** per ottenere gli **score**. Tutti *questi dati vengono salvati* in formato **.csv e .txt** nella **cartella /report** e le **immagini** nella cartella **/img**



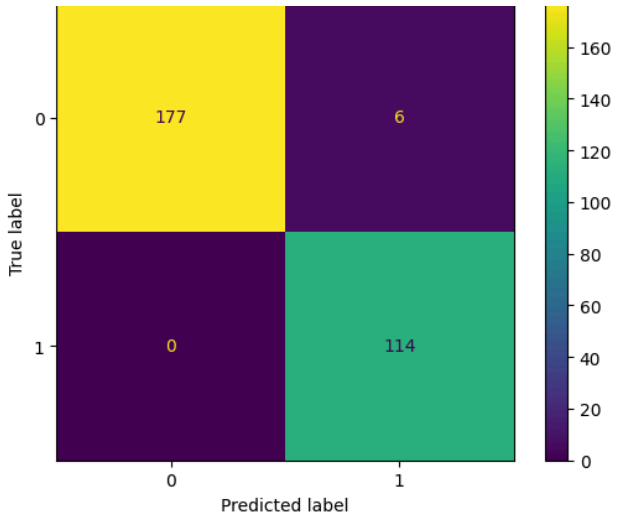
	ACCURACY	BEST PARAMS	BEST ESTIMATOR
LR	0.97	'C': 0.0005, 'class_weight': 'balanced', 'penalty': 'l2'	LogisticRegression(C=0.0005, class_weight='balanced', solver='saga')
KNN	0.98	'algorithm': 'auto', 'n_neighbors': 1, 'weights': 'uniform'	KNeighborsClassifier(n_neighbors=1)
SVC	0.96	'C': 100.0, 'class_weight': 'balanced', 'gamma': 0.001	SVC(C=100.0, class_weight='balanced', gamma=0.001)
DT	0.99	'class_weight': 'balanced', 'criterion': 'gini', 'max_features': 'sqrt'	DecisionTreeClassifier(class_weight='balanced', max_features='sqrt')
MLP	0.98	{'activation': 'tanh', 'solver': 'lbfgs'}	MLPClassifier(activation='tanh', solver='lbfgs')

Cross validation e Confusion Matrix:

STACKING ENSEMBLE
FINAL SCORE
Il risultato finale dello stacking ensemble è il seguente:

TRAINING
Training Accuracy : 1.00000

TEST
Test Accuracy : 0.97643
Test Precision : 0.97107
Test Recall : 0.98087
Test F1 Score : 0.97536



Classification Report:

	precision	recall	f1-score	support
0	1.00	0.96	0.98	183
1	0.94	1.00	0.97	114
accuracy			0.98	297
macro avg	0.97	0.98	0.98	297
weighted avg	0.98	0.98	0.98	297

MODELLI TESTATI SINGOLARMENTE

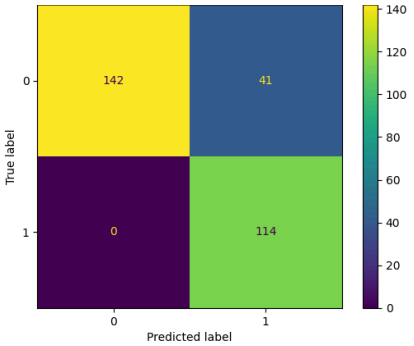
Si prova anche a **procedere singolarmente con i modelli**, anziché affidarsi ad uno stacking ensemble, per valutare se questa scelta convenga o meno.

PRIMO MODELLO TESTATO SINGOLARMENTE - Logistic Regression

Cross validation e Confusion Matrix:

TRAINING
Training Accuracy : 0.91000

TEST
Test Accuracy : 0.86195
Test Precision : 0.86774
Test Recall : 0.88798
Test F1 Score : 0.86071

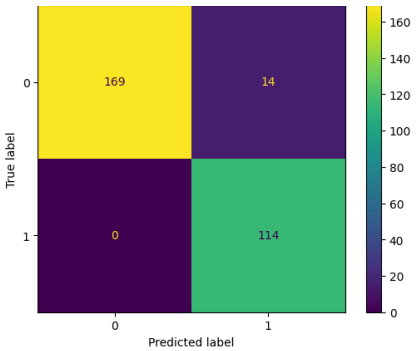


SECONDO MODELLO TESTATO SINGOLARMENTE - KNeighborsClassifier

Cross validation e Confusion Matrix:

TRAINING
Training Accuracy : 1.00000

TEST
Test Accuracy : 0.95286
Test Precision : 0.94531
Test Recall : 0.96175
Test F1 Score : 0.95119

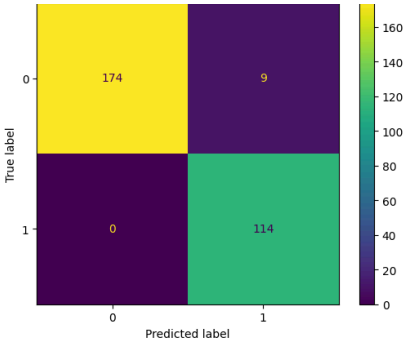


TERZO MODELLO TESTATO SINGOLARMENTE - *SVC*

Cross validation e Confusion Matrix:

TRAINING
Training Accuracy : 1.00000

TEST
Test Accuracy : 0.96970
Test Precision : 0.96341
Test Recall : 0.97541
Test F1 Score :0.96841

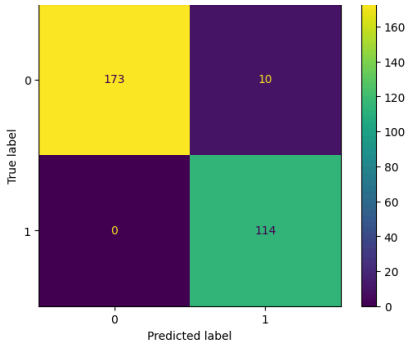


QUARTO MODELLO TESTATO SINGOLARMENTE - *DecisionTreeClassifier*

Cross validation e Confusion Matrix:

TRAINING
Training Accuracy : 1.00000

TEST
Test Accuracy : 0.96633
Test Precision : 0.95968
Test Recall : 0.97268
Test F1 Score : 0.96495

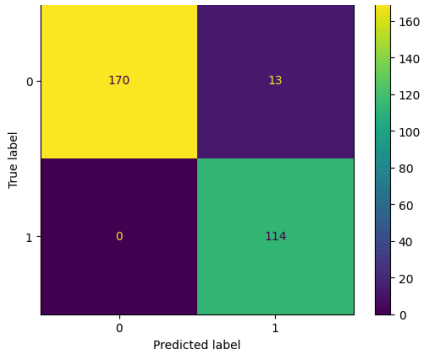


QUINTO MODELLO TESTATO SINGOLARMENTE - MLPClassifier

Cross validation e Confusion Matrix:

TRAINING
Training Accuracy : 1.00000

TEST
Test Accuracy : 0.95623
Test Precision : 0.94882
Test Recall : 0.96448
Test F1 Score : 0.95462



CONSIDERAZIONI FINALI

In definitiva, si è dimostrato empiricamente che lo **Stacking Ensemble ha prestazioni migliori**, (anche se di poco) rispetto ai singoli modelli selezionati singolarmente, con una **confusion matrix più performante** ed i seguenti valori:

Test Accuracy : 0.97643
Test Precision : 0.97107
Test Recall : 0.98087
Test F1 Score : 0.97536